Weights or Skills?

A Survey of Robot-Learning Techniques—from Action-Predicting Weights to Robots that Write their Own Skills

GAYTRI JENA, UC Berkeley, USA
KAPIL WANASKAR, Canva Research, USA
VINIJA JAIN, Google, USA
AMAN CHADHA, Google DeepMind, USA
VASU SHARMA, PocketFM, USA
AMITAVA DAS, Pragya Lab, BITS Pilani Goa, India

Robot learning is splitting into two bets: policies that bake competence into *frozen weights*—vision-language-action (VLA) models—and agents that write and refine their own *executable skills* as code. This survey organises the field around that axis, **weights or skills**, and makes its central contribution a deep-dive that arranges code-as-policy methods by their *degree of self-improvement*: from zero-shot program synthesis, through closed-loop self-repair and persistent skill memory, to the near-empty cell where execution feedback, skill memory, and evolutionary search combine into one open-ended loop. We anchor that frontier on ASPIRE and its real-world sibling ENPIRE, and we map the complementary "skills" pole—from unsupervised reinforcement-learning skill discovery to large-language-model skill libraries—showing that skills come in an RL flavour and a code flavour, of which only the code flavour self-improves without gradients. Finally, we connect the taxonomy to the emerging *skill economy*: commercial robot-skill marketplaces now distribute one-tap skills across robots, but ship only static playback, and we lay out the open problems—adaptation, cross-embodiment portability, provenance, safety verification, composition, and standardisation—that separate a store of animations from a store of capabilities. The survey covers roughly seventy-five systems across six technique families and provides a canonical taxonomy figure for each.

CCS Concepts: • **Computing methodologies** → **Robotic planning**; *Machine learning*.

Additional Key Words and Phrases: robot learning, code-as-policy, self-improvement, vision-language-action models, skill libraries, physical AI, survey

1 INTRODUCTION

Two paradigms now dominate robot learning. *Vision-language-action* (VLA) models predict actions from frozen weights; *code-as-policy* agents instead write executable programs and improve them from experience [32, 39]. This survey is organized around that fault line.

Why now: the skill stack. The app store for robot skills has arrived—Unitree UniStore ships one-tap, cross-model motion downloads [55]. But every skill in it is *canned playback*: the least-capable point on our map. The skill stack has three layers—**(1) Build** (how skills are created, §2–§3), **(2) Distribute** (marketplaces and hubs), and **(3) Adapt** (on-device self-improvement, §3.1). UniStore is layer 2 over static layer-1 skills; the promised value—"adjust grip per fruit so nothing bruises"—lives in layer 3, which no shipped skill does today.

Contributions. (i) a taxonomy of robot-learning techniques organized by *weights vs. skills* (§2); (ii) a deep-dive arranging code-as-policy by *degree of self-improvement* and isolating the un-surveyed §3.1.5 cell (§3.1); (iii) a companion map of the "skills" pole (§3.4); and (iv) an open-problems agenda for the emerging skill economy (§4).

Authors' addresses: Gaytri Jena, UC Berkeley, Berkeley, CA, USA; Kapil Wanaskar, Canva Research, USA; Vinija Jain, Google, USA; Aman Chadha, Google DeepMind, USA; Vasu Sharma, PocketFM, USA; Amitava Das, Pragya Lab, BITS Pilani Goa, India.

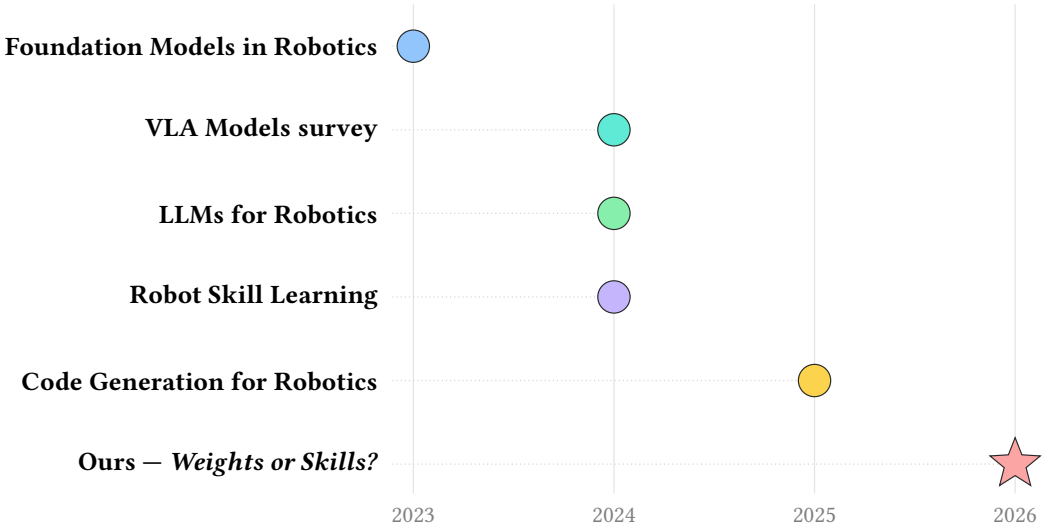

Table 1. Related surveys on a timeline. Prior surveys (2023–2025) each cover part of the landscape—foundation models, VLA policies, LLM planning, skill acquisition, code generation—but none organizes code-as-policy by *degree of self-improvement*. **Ours** (★, 2026) isolates the feedback + memory + search cell (§3.1.5) that ASPIRE occupies.

Related surveys. Table 1 places the closest prior surveys on a timeline. Each covers part of the landscape—foundation models, VLA policies, LLM-based planning, skill acquisition, or code generation—but none organises code-as-policy by *degree of self-improvement* or isolates the §3.1.5 cell, which is the gap this survey fills.

2 A TAXONOMY OF ROBOT-LEARNING TECHNIQUES

Figure 1 organizes the field into six branches. The dividing question is *what ships*: frozen weights (§3.2) or executable skills (§3.1). We tag mechanisms by **F**eedback, **S**earch, and **M**emory throughout.

Table 2 compares all systems in the map on what each *ships*, where it is evaluated, and whether it improves from its own experience.

Table 2. Comparison of all ∼47 systems in the main taxonomy (Figure 1), grouped by family. **Ships**: **code**/**weights**/**reward**/**skill**/**policy**/**bench**. **Eval**: real/sim/game. **Self-impr.**: ✔ learns from its own experience, ~ partially, ✘ no, – n/a.

System	Year	Venue	Ships	Eval	Self-impr.
§3.1 Code-as-policy for robots					**code**
Code-as-Policies [32]	2023	ICRA	**code**	real	✘
ProgPrompt [53]	2023	ICRA	**code**	real	✘
VoxPoser [20]	2023	CoRL	**code**	real	✘
RoboCodeX [43]	2024	ICML	**code**	sim	✘
Instruct2Act [19]	2023	arXiv	**code**	sim	✘
RoboPro [65]	2025	arXiv	**code**	real	✘
RoboCoder [27]	2024	arXiv	**code**	sim	✘
CaP-X [13]	2026	arXiv	**code**	sim	✔
RoboClaw [29]	2026	arXiv	**code**	real	✔
ASPIRE [39]	2026	arXiv	**code**	real	✔

Table 2 (continued)

System	Year	Venue	Ships	Eval	Self-impr.
ENPIRE [64]	2026	arXiv	code	real	✔
§3.2 End-to-end VLA / generalist policies					weights
RT-1 [5]	2023	RSS	weights	real	✘
RT-2 [6]	2023	CoRL	weights	real	✘
Octo [15]	2024	RSS	weights	real	✘
OpenVLA [23]	2024	CoRL	weights	real	✘
RoboFlamingo [30]	2024	ICLR	weights	sim	✘
CogACT [28]	2024	arXiv	weights	real	✘
SpatialVLA [49]	2025	arXiv	weights	real	✘
π_0 [3]	2024	arXiv	weights	real	✘
$\pi_{0.5}$ [2]	2025	arXiv	weights	real	✘
GR00T N1 [44]	2025	arXiv	weights	real	✘
Gemini Robotics [14]	2025	arXiv	weights	real	✘
§3.3 LLM-authored rewards / curricula					reward
Eureka [40]	2024	ICLR	reward	sim	✔
DrEureka [41]	2024	RSS	reward	real	✔
Text2Reward [67]	2024	ICLR	reward	sim	~
Language-to-Rewards [70]	2023	CoRL	reward	real	~
Eurekaverse [34]	2024	CoRL	reward	sim	✔
RoboGen [60]	2024	ICML	reward	sim	✔
Auto MC-Reward [26]	2024	CVPR	reward	game	✔
§3.4 Robot skill libraries & lifelong learning					skill
Voyager [58]	2023	arXiv	skill	game	✔
LOTUS [57]	2024	ICRA	skill	sim	✔
LRLL [54]	2024	ICRA	skill	sim	✔
BOSS [73]	2023	CoRL	skill	sim	✔
SPRINT [72]	2024	ICRA	skill	sim	~
Uni-Skill [66]	2026	ICRA	skill	real	✔
SkillFlow [74]	2026	arXiv	skill	text	✔
§3.5 Sim-to-real & cross-embodiment transfer					policy
Open X-Embodiment [45]	2024	ICRA	policy	real	✘
CrossFormer [10]	2024	CoRL	policy	real	✘
RoboCat [4]	2023	TMLR	policy	real	✔
Mirage [9]	2024	RSS	policy	real	✘
§3.6 Embodied benchmarks & simulators					bench
LIBERO [36]	2023	NeurIPS	bench	sim	–
Robosuite [78]	2020	arXiv	bench	sim	–
BEHAVIOR-1K [25]	2024	arXiv	bench	sim	–
Meta-World [69]	2019	CoRL	bench	sim	–
RLBench [22]	2020	RA-L	bench	sim	–
ManiSkill2 [16]	2023	ICLR	bench	sim	–
CALVIN [42]	2022	RA-L	bench	sim	–

3 THE TECHNIQUE FAMILIES

We now examine each of the six branches of the taxonomy (Figure 1) in turn. We begin with the code-as-policy camp (§3.1)—the focus of this survey and the only family we organise by *degree of self-improvement*—and then survey the weights-based (§3.2), reward-synthesis (§3.3), skill-library (§3.4), transfer (§3.5), and benchmark (§3.6) families that surround it.

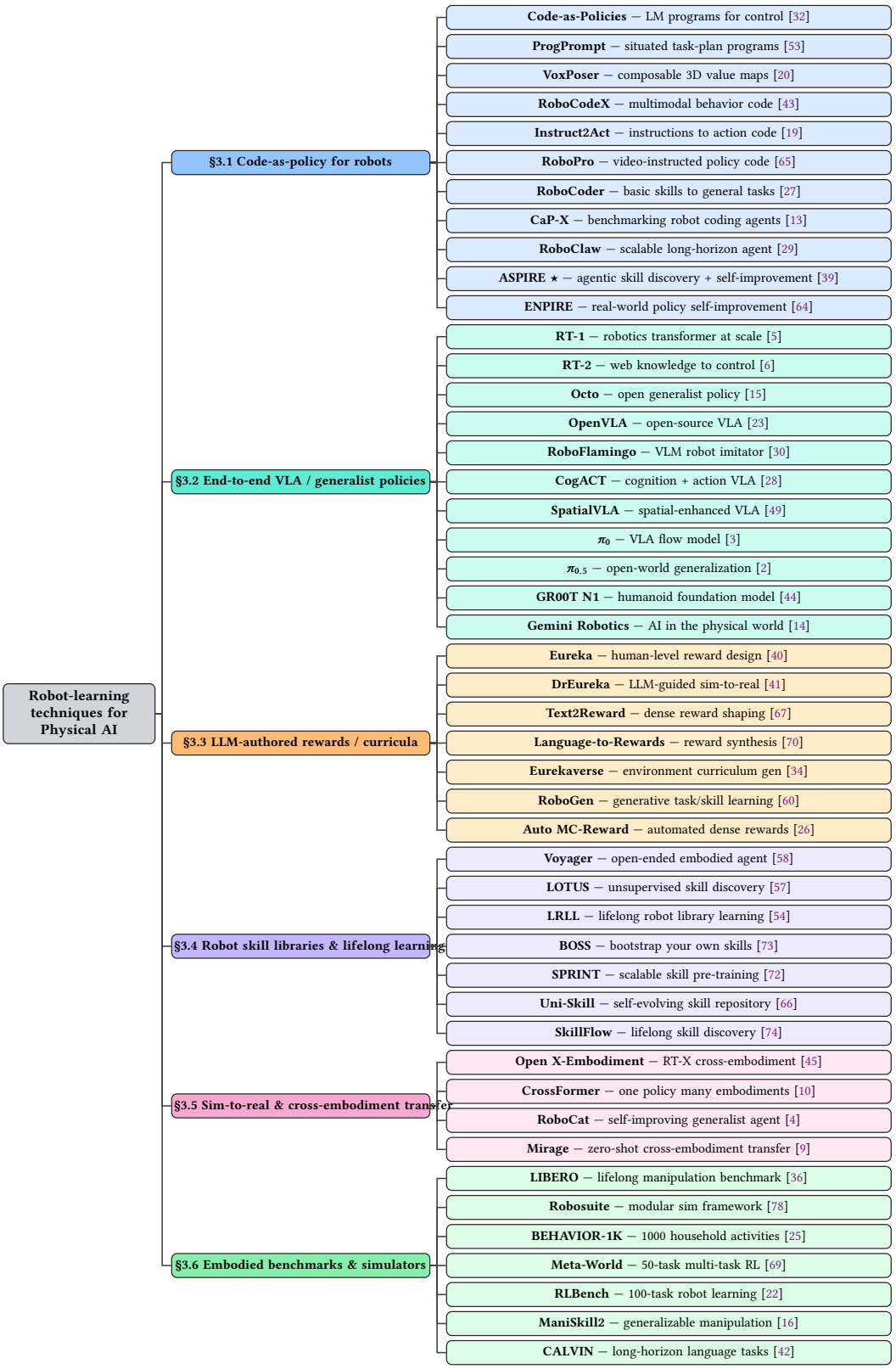

Fig. 1. Taxonomy of robot-learning techniques for Physical AI (~47 systems). The fault line: §3.1 ships *code* and §3.2 ships *weights*. **ASPIRE** ⋆ anchors the self-improving code-as-policy frontier (§3.1).

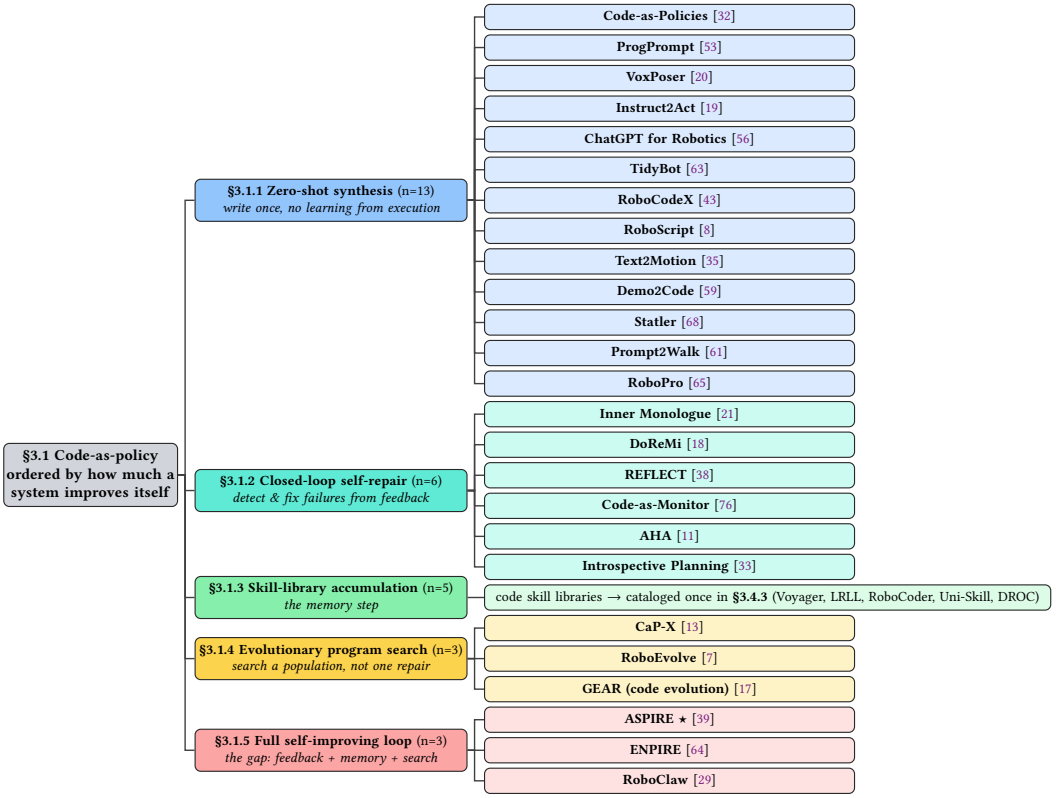

Fig. 2. The self-improvement funnel within code-as-policy ($13 \to 6 \to 5 \to 3 \to 3$), one colour per sub-section. As the demand rises (write once → repair → remember → search → all three), occupants thin out; §3.1.5—feedback + memory + search combined—is nearly empty, and only **ASPIRE** ⋆ was built for it end-to-end.

3.1 Code-as-Policy: Robots that Write their Own Skills

The organising contribution of this survey is to arrange code-as-policy methods not by application or robot embodiment, but by *how much of their own competence a system produces at run time.* Concretely, we ask five questions in sequence: does the agent (i) write control code at all, (ii) repair that code from execution feedback within a task, (iii) remember validated code across tasks, (iv) search a population of programs rather than following a single trajectory of repairs, and (v) close all of these into one open-ended loop? Each "yes" is a rung on the ladder of Figure 2, and the population thins sharply toward the top: of the thirteen zero-shot systems, only a handful ever reach the combined feedback–memory–search regime. The novelty of the survey is naming this progression and showing that the top rung is nearly empty.

3.1.1 Zero-shot synthesis. The foundational move is to treat the language model as a *program synthesiser* over a fixed library of perception and control primitives. *Code-as-Policies* [32] showed that an LLM prompted with an API and a natural-language instruction can emit executable Python that composes those primitives, recursively defining undefined functions and parameterising control with arithmetic and feedback logic. *ProgPrompt* [53] situates the generated program in the scene by exposing available actions and objects as importable symbols, while *VoxPoser* [20] has the model write code that *composes 3D value maps* for a motion planner rather than calling fixed skills,

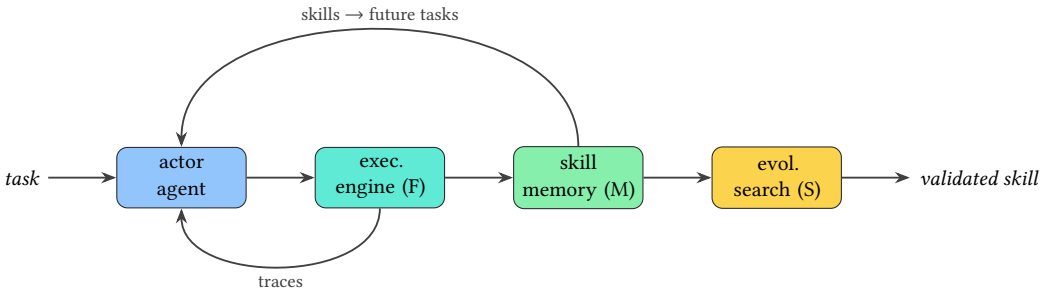

Fig. 3. The full self-improving loop (§3.1.5): feedback (F) + memory (M) + search (S) combine into one open-ended loop—the ASPIRE cell. VLAs (§3.2) have no such loop.

loosening the dependence on a hand-authored API. Subsequent work broadened the input modality and task horizon—*RoboCodeX* [43] conditions on multimodal observations, *Instruct2Act* [19] maps instructions to perception–action code, and video- and demonstration-conditioned variants such as *RoboPro* [65], *Demo2Code* [59] and *Text2Motion* [35] recover programs from richer context. *Statler* [68] maintains an explicit world state across steps, and *TidyBot* [63] and *ChatGPT for Robotics* [56] demonstrate personalisation and prompt design for real hardware. What unites this rung, and limits it, is that the program is written *once*: there is no channel from execution back to the synthesiser, so a plan that fails at run time simply fails.

3.1.2 Closed-loop self-repair. The second rung adds a within-task feedback loop. *Inner Monologue* [21] feeds success detectors, scene descriptions and human feedback back into the model as language, letting it replan when a step fails. *DoReMi* [18] detects misalignments between plan and execution and triggers recovery, and *REFLECT* [38] builds a hierarchical, multimodal summary of past interaction that an LLM queries to explain a failure and propose a correction. More recent systems push the monitor into the perception loop—*Code-as-Monitor* [76] compiles spatio-temporal constraints into vision-language checks, and *AHA* [11] trains a model to reason explicitly about failure modes. The novelty of this rung is the closed loop, but it is a *short* loop: corrections are discarded at task boundaries, so nothing accumulates.

3.1.3 Skill-library accumulation. The third rung makes the loop persistent by writing validated code into a growing library that later tasks retrieve and compose. Because this "memory" axis is a full technique family in its own right, we catalogue its systems once, in §3.4 (§3.4.3)—*Voyager* [58] is the canonical example, curating an ever-growing library of verified skill programs with when-to-apply guards, while related methods distil language corrections into retrievable knowledge for future tasks [71]. The key distinction from §3.1.2 is temporal scope: competence compounds *across* tasks rather than being rebuilt each time.

3.1.4 Evolutionary program search. The fourth rung replaces a single chain of repairs with population-based search over programs. Rather than iteratively debugging one candidate, the agent maintains several, scores them by execution, and mutates the best. *CaP-X* [13] provides an interactive gym and benchmark for exactly this style of coding agent, and *RoboEvolve* [7] couples a planner and a learned simulator into a co-evolutionary loop that mines near-miss failures to stabilise search. The novelty is the shift from *repair* to *search*: exploration is explicit and parallel, which trades compute for a better chance of escaping local failures.

3.1.5 The full self-improving loop. The top rung combines all three mechanisms—execution feedback (F), a persistent skill memory (M), and evolutionary search (S)—into one open-ended loop (Figure 3). *ASPIRE* [39] instantiates this with a robot execution engine that emits per-primitive multimodal traces, a skill library of validated code with guards, and a population search over candidate programs, so that experience compounds and transfers across tasks and embodiments. Its real-world sibling *ENPIRE* [64] closes the same loop on physical hardware, using automatic reset, parallel rollouts, and log-driven algorithm revision to drive a coding agent to near-perfect success on dexterous tasks. This cell—feedback *and* memory *and* search—is what no prior survey isolates, and it is nearly empty: it is the flag this survey plants.

Table 3 makes the funnel explicit, tabulating each system's use of feedback (F), memory (M), and search (S).

Table 3. The 30 code-as-policy systems of Figure 2 on the three self-improvement mechanisms—**F**eedback, **M**emory, **S**earch—which are determined by the rung. ✔ present, ✘ absent. **Self-impr.**: ✔ full loop, ~ partial, ✘ none.

System	Year	F	M	S	Self-impr.
§3.1.1 Zero-shot synthesis				F✘	M✘ S✘
Code-as-Policies [32]	2023	✘	✘	✘	✘
ProgPrompt [53]	2023	✘	✘	✘	✘
VoxPoser [20]	2023	✘	✘	✘	✘
Instruct2Act [19]	2023	✘	✘	✘	✘
ChatGPT for Robotics [56]	2023	✘	✘	✘	✘
TidyBot [63]	2023	✘	✘	✘	✘
RoboCodeX [43]	2024	✘	✘	✘	✘
RoboScript [8]	2024	✘	✘	✘	✘
Text2Motion [35]	2023	✘	✘	✘	✘
Demo2Code [59]	2023	✘	✘	✘	✘
Statler [68]	2024	✘	✘	✘	✘
Prompt2Walk [61]	2024	✘	✘	✘	✘
RoboPro [65]	2025	✘	✘	✘	✘
§3.1.2 Closed-loop self-repair				F✔	M✘ S✘
Inner Monologue [21]	2022	✔	✘	✘	✘
DoReMi [18]	2024	✔	✘	✘	✘
REFLECT [38]	2023	✔	✘	✘	✘
Code-as-Monitor [76]	2025	✔	✘	✘	✘
AHA [11]	2024	✔	✘	✘	✘
Introspective Planning [33]	2024	✔	✘	✘	✘
§3.1.3 Skill-library accumulation				F✘	M✔ S✘
Voyager [58]	2023	✘	✔	✘	~
LRLL [54]	2024	✘	✔	✘	~
RoboCoder [27]	2024	✘	✔	✘	~
Uni-Skill [66]	2026	✘	✔	✘	~
DROC [71]	2024	✔	✔	✘	~
§3.1.4 Evolutionary program search				F✔	M✘ S✔
CaP-X [13]	2026	✔	✘	✔	~
RoboEvolve [7]	2026	✔	✘	✔	~
Code Evolution (GEAR) [17]	2026	✔	✘	✔	~
§3.1.5 Full self-improving loop				F✔	M✔ S✔
ASPIRE [39]	2026	✔	✔	✔	✔
ENPIRE [64]	2026	✔	✔	✔	✔
RoboClaw [29]	2026	✔	✔	✔	✔

3.2 End-to-End Vision-Language-Action Models

The dominant alternative to writing code is to regress actions directly from pixels and language with a single large network, so that competence lives in *frozen weights* rather than in an inspectable program. Architecturally these vision-language-action (VLA) models share a backbone–action-head decomposition: a vision-language backbone encodes the scene and instruction, and a pluggable action head maps that representation to motor commands [3, 23].

The lineage begins with *RT-1* [5], a transformer trained on large-scale real robot data that discretises continuous actions into categorical bins. *RT-2* [6] then co-fine-tunes an Internet-pretrained vision-language model on robot trajectories, emitting actions as text tokens and inheriting semantic generalisation from web data. Open reproductions followed: *Octo* [15] and *OpenVLA* [23]—the latter a 7B model with a DINOv2/SigLIP vision stack—are trained on the pooled Open X-Embodiment corpus (§3.5), and *RoboFlamingo* [30] adapts a vision-language model into an imitation policy.

A second design axis is the *action representation*. Discrete token heads (RT-1/RT-2) are simple but coarse; recent systems favour continuous heads. π_0 [3] attaches a *flow-matching* head to a PaLI-Gemma backbone for smooth, high-frequency bimanual control, and $\pi_{0.5}$ [2] extends it toward open-world generalisation; *CogACT* [28] separates a cognition module from a diffusion-transformer action module, and *SpatialVLA* [49] injects 3D spatial encodings. Flow- and diffusion-based heads trade a few extra parameters for far fewer inference steps and better high-degree-of-freedom precision.

At the largest scale, foundation-model efforts target whole platforms: *GR00T N1* [44] pairs a slow System-2 vision-language reasoner with a fast System-1 diffusion transformer that produces motor actions at high rate for humanoids, and *Gemini Robotics* [14] builds a VLA together with an embodied-reasoning model on the Gemini backbone. Across all of these, the defining property—and the reason we treat them as the *foil* for this survey—is that there is *no loop and no code*: competence is acquired by scaling data and parameters, generalises by interpolation rather than search, and cannot be edited, audited, or recombined after training.

3.3 Reward and Curriculum Synthesis

A third family keeps the language model in the role of programmer but points it at a different artefact: instead of policy code, it writes the *reward*, *environment*, or *curriculum* that a reinforcement-learning loop then compiles into a policy. A feedback loop exists—the model reads back training statistics and revises—but what ships is still weights, which is why we separate this family from the self-improving code camp (§3.1).

Eureka [40] is the canonical instance: given unmodified environment source code and a task description, a coding LLM zero-shot generates an executable reward function, then improves it by *evolutionary search* guided by *reward reflection*—a textual summary of training statistics—exceeding expert human rewards on 83% of a 29-task IsaacGym suite. *DrEureka* [41] extends the idea to sim-to-real by also synthesising domain-randomisation ranges, achieving zero-shot real-world locomotion. *Text2Reward* [67] generates dense reward programs grounded in a compact environment representation and refines them from human feedback, matching or beating expert-written rewards on most of a seventeen-task suite, while *Language to Rewards* [70] uses the reward function as the interface between an LLM and a MuJoCo model-predictive controller.

Beyond the reward itself, the same generative recipe is applied to the *training environment*. *Eurekaverse* [34] has an LLM propose a progressively harder curriculum of environments—learning parkour skills that transfer to a real robot—and *RoboGen* [60] runs a propose-generate-learn cycle that fabricates tasks, scenes, and supervision in simulation to unlock effectively unlimited training data; *Auto MC-Reward* [26] automates dense rewards for open-world Minecraft. The through-line

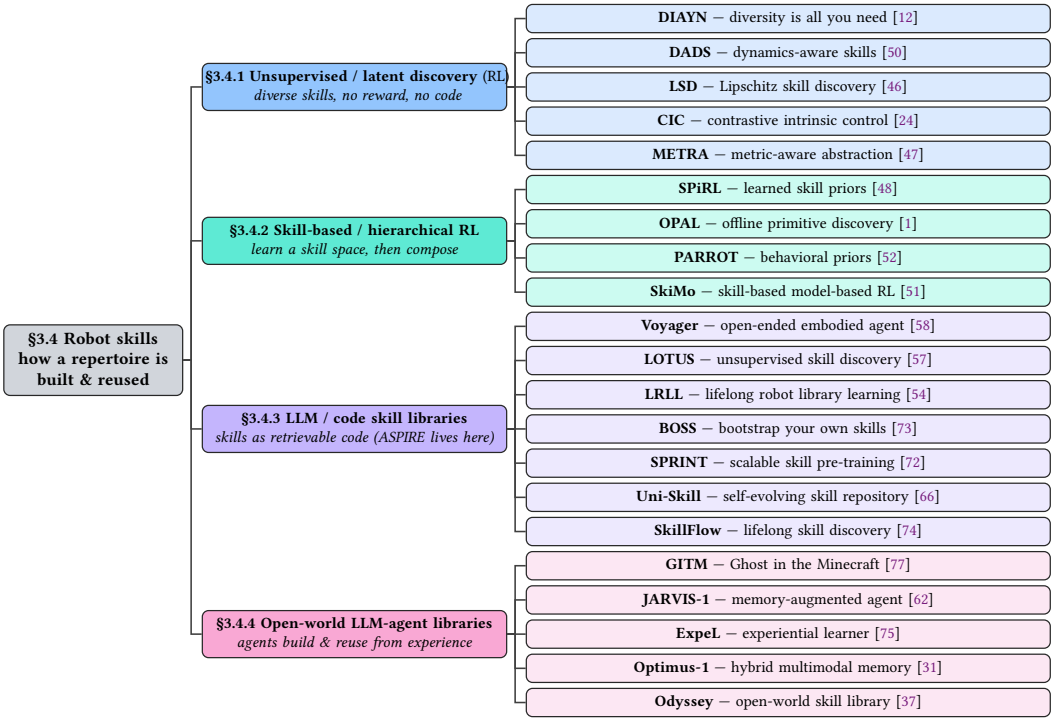

Fig. 4. The "skills" pole, one colour per sub-section. §3.4.1–2 build skills as *RL policies* (mature, included as landscape); §3.4.3–4 build them as *retrievable code*, where ASPIRE's self-improving §3.1 lives.

is that language becomes a bridge from high-level intent to low-level control *via* an RL optimiser, rather than producing an executable, reusable skill directly.

3.4 Skill Libraries: How a Repertoire is Built

Where §3.1 asked *how* a code skill improves, this family asks the prior question of *how a repertoire of skills is built at all*. Figure 4 organises the answer along one axis—skills learned as latent reinforcement-learning policies versus skills stored as retrievable code—and only the code flavour self-improves without gradients, which links this family back to §3.1.

3.4.1 Unsupervised and latent skill discovery. The oldest lineage learns a diverse set of skills with *no task reward and no code*, by maximising an information-theoretic objective. *DIAYN* [12] maximises the mutual information between a latent skill code and the visited states, yielding distinguishable behaviours; *DADS* [50] makes the discovered skills dynamics-aware and hence usable for model-based control; and *LSD* [46], *CIC* [24], and *METRA* [47] push toward more dynamic, far-reaching, and scalable skills through Lipschitz constraints, contrastive objectives, and metric-aware abstractions respectively.

3.4.2 Skill-based and hierarchical reinforcement learning. A second lineage extracts a continuous *skill space* from offline data and then plans or learns within it. *SPiRL* [48] learns a skill embedding together with a skill prior that accelerates downstream RL; *OPAL* [1] discovers temporally extended primitives for offline RL; *PARROT* [52] learns an invertible behavioural prior; and *SkiMo* [51] learns a skill dynamics model so that planning happens directly in skill space.

3.4.3 LLM and code skill libraries. The code flavour—where the self-improving methods of §3.1 live—stores skills as retrievable programs. *Voyager* [58] curates an ever-growing library of verified code skills in Minecraft; *LOTUS* [57] performs continual imitation learning through unsupervised skill discovery; *LRLL* [54] bootstraps a composable robot library with language models; and *BOSS* [73] and *SPRINT* [72] grow skill repertoires by LLM-guided bootstrapping and instruction relabelling. The most recent entries close the loop between video and code: *Uni-Skill* [66] builds a self-evolving skill repository from unstructured robot video, and *SkillFlow* [74] benchmarks lifelong discovery, patching, and reuse of a skill library.

3.4.4 Open-world LLM-agent libraries. Finally, embodied LLM agents accumulate skills from open-ended experience. *GITM* [77] and *JARVIS-1* [62] pair a planner with text or multimodal memory in Minecraft; *ExpeL* [75] extracts reusable insights from a stream of trials; *Optimus-1* [31] adds a hybrid multimodal memory for long-horizon tasks; and *Odyssey* [37] equips an agent with an open-world library of primitive and compositional skills. These systems foreshadow the marketplace dynamics of §4: a library that grows with use.

Table 4 contrasts the four sub-families on skill form, whether they are gradient-trained or LLM-authored, whether they keep a persistent library, and their domain.

Table 4. The 21 skill-building systems of Figure 4, by sub-family. **Form**: latent-RL/skill-space/code-lib/agent-lib. **Grad.**: gradient-trained (✔) vs LLM-authored (✘). **Lib.**: persistent retrievable library. **Domain**: sim/game/real/text.

System	Year	Form	Grad.	Lib.	Domain
§3.4.1 Unsupervised / latent discovery (RL)					
DIAYN [12]	2019	latent-RL	✔	✘	sim
DADS [50]	2020	latent-RL	✔	✘	sim
LSD [46]	2022	latent-RL	✔	✘	sim
CIC [24]	2022	latent-RL	✔	✘	sim
METRA [47]	2024	latent-RL	✔	✘	sim
§3.4.2 Skill-based / hierarchical RL					
SPiRL [48]	2020	skill-space	✔	✘	sim
OPAL [1]	2021	skill-space	✔	✘	sim
PARROT [52]	2021	skill-space	✔	✘	sim
SkiMo [51]	2022	skill-space	✔	✘	sim
§3.4.3 LLM / code skill libraries					
Voyager [58]	2023	code-lib	✘	✔	game
LOTUS [57]	2024	code-lib	✘	✔	sim
LRLL [54]	2024	code-lib	✘	✔	sim
BOSS [73]	2023	code-lib	✘	✔	sim
SPRINT [72]	2024	code-lib	✘	✔	sim
Uni-Skill [66]	2026	code-lib	✘	✔	real
SkillFlow [74]	2026	code-lib	✘	✔	text
§3.4.4 Open-world LLM-agent libraries					
GITM [77]	2023	agent-lib	✘	✔	game
JARVIS-1 [62]	2023	agent-lib	✘	✔	game
ExpeL [75]	2024	agent-lib	✘	✔	text
Optimus-1 [31]	2024	agent-lib	✘	✔	game
Odyssey [37]	2024	agent-lib	✘	✔	game

3.5 Sim-to-Real and Cross-Embodiment Transfer

A skill is only useful if it moves—across the simulation-to-reality gap and across robot bodies. The enabling artefact is scale: *Open X-Embodiment* [45] pools sixty datasets from twenty-one institutions into over a million trajectories spanning twenty-two embodiments and hundreds of skills, and shows that RT-X models trained on the mixture exhibit positive transfer and emergent capabilities across platforms.

Given such data, a single network can control very different bodies. *CrossFormer* [10] trains one transformer on 900K trajectories across twenty embodiments—arms, wheeled robots, quadrotors, and quadrupeds—*without* manually aligning observation or action spaces, and matches specialist policies tailored to each robot. Other work attacks transfer at test time rather than through data: *Mirage* [9] achieves *zero-shot* cross-embodiment transfer by "cross-painting"—masking the target robot and inpainting the source robot at the same pose so the policy sees a familiar body—and bridging the control gap with forward dynamics.

The most suggestive point for this survey is *RoboCat* [4], a goal-conditioned multi-embodiment agent that adapts to a new task from as few as a hundred demonstrations and then *generates its own data* for the next training round, forming a rudimentary self-improvement loop of the kind §3.1 makes explicit. This is also the marketplace question of §4 in disguise: a skill advertised for the G1, H1, B2, and Go2 platforms must survive exactly the cross-embodiment gap these methods study.

3.6 Benchmarks and Evaluation

Progress in the families above is measured on a shared set of simulated benchmarks, most of which report a single-number success rate. *LIBERO* [36] targets lifelong manipulation and knowledge transfer across task suites; *Meta-World* [69] offers fifty manipulation tasks for multi-task and meta-reinforcement learning; *RLBench* [22] provides a hundred vision-based tasks with demonstrations; and *ManiSkill2* [16] adds GPU-parallel simulation over twenty task families. *Robosuite* [78] is the modular MuJoCo framework underlying many of these, and *CALVIN* [42] evaluates long-horizon, language-conditioned control by the average number of sub-tasks completed in a chain. At the high-realism end, *BEHAVIOR-1K* [25] specifies a thousand everyday household activities in a photorealistic simulator, scoring both success and efficiency.

Table 5 contrasts one exemplar per camp on the axes that separate them. The recurring gap—and a direct motivation for the open problems of §4—is that these suites measure *one-shot competence*: they report whether a policy succeeds, not whether it *improves with experience*. No standard benchmark yet plots held-out success as a function of accumulated interaction, which is exactly the quantity the self-improvement ladder of §3.1 is designed to raise.

4 THE SKILL ECONOMY: OPEN PROBLEMS

The arrival of a commercial marketplace for robot skills—Unitree's UniStore ships one-tap, cross-model motion packages [55]—turns several research questions from hypothetical into pressing. We frame them as the open agenda of this survey.

The static-to-adaptive gap. Every skill shipped today is, in effect, replayed: a motion package or a fixed program is installed and executed without perceiving whether *this* kitchen, gripper, or object differs from the one it was authored for. The techniques of §3.1—within-task repair, persistent memory, and search—are precisely what would let a downloaded skill *adapt* on the target robot. Closing this gap is the difference between a store of animations and a store of capabilities, and it is the direct application of the §3.1.5 loop to a distribution setting where the base skill is given rather than discovered.

Dimension	ASPIRE [39] *code+skills*	π_0 [3] *VLA weights*	Code-as-Pol. [32] *code, no mem.*	Eureka [40] *reward*
What it outputs	skill code+lib	action chunks	policy code	reward code
How it improves	F+M+S loop	gradient pretrain	– one-shot	RL+reflection
Gradient training?	✘	✔	✘	✔
Persistent skill library?	✔	✘	✘	✘
Failure feedback signal	exec. traces	–	–	train. stats
Search strategy	evolutionary	–	–	–
Real-robot validation	✔	✔	✔	✔
Continual / open-ended?	✔	✘	✘	✘
Beyond fixed APIs?	✔	✔	✘	–
Interpretable / editable?	✔	✘	✔	✔

Table 5. Capability matrix, one exemplar per camp; each method header links to its entry in the References. Only ASPIRE combines a persistent skill library with feedback- and search-driven self-improvement. ✔ = yes, ✘ = no, – = n/a.

Cross-embodiment portability. UniStore advertises a single skill running across the G1 and H1 humanoids and the B2/Go2 quadrupeds—bodies with different morphologies, action spaces, and dynamics. This is the cross-embodiment transfer problem (§3.5) at deployment scale: without a shared action interface [45] or explicit embodiment adaptation, a package tuned for one platform has no guarantee of correctness on another. Formal portability—what must hold for a skill to be certified on a new body—remains open.

Provenance and trust. A marketplace invites third-party, crowdsourced uploads, including raw motion-capture data and code from unknown authors. Robot skills act on the physical world, so provenance (who authored a skill, from what data, validated how) becomes a safety property, not merely a metadata nicety. There is no accepted standard for signing, attesting, or auditing a robot skill's origin.

Safety verification. Vendors promise that "every skill is scanned, tested, and verified," but for a code-as-policy skill this is undecidable in general and hard in practice: the skill's effect depends on the scene it meets. Practical questions include which pre-conditions a skill must declare, what sandboxed or simulated screening it must pass, and how run-time monitors (cf. *Code-as-Monitor* [76]) can bound behaviour on unseen inputs.

Skill composition. Downloading two skills should ideally yield a third: "make coffee" plus "clear the table" composed into a morning routine. Composition requires shared interfaces and a calculus over pre-/post-conditions; today's motion packages are opaque and do not compose, and code skills compose only within a single authoring system.

Portability standards and skill ontologies. Reuse at scale needs a shared vocabulary: declared pre-conditions, expected effects, and embodiment-capability profiles, with relations such as *requires*, *provides*, and *substitutable-by*. Prior work on skill ontologies and hardware-level reusability points the way, but no cross-vendor standard exists, which is what a genuine ecosystem would require.

Local versus shared libraries. Finally, the field must reconcile two notions of "skill library" that this survey has kept distinct: the *self-built* library a robot grows from its own experience (ASPIRE, §3.1) and the *shared* library a robot downloads from a marketplace (UniStore). The interesting regime

is the hybrid—an agent that both contributes to and draws from a commons—and its incentives, quality control, and feedback dynamics are entirely unstudied.

5 LIMITATIONS OF THIS SURVEY

We state the boundaries of the survey explicitly. First, our organising axis—*degree of self-improvement*—is deliberately code-centric; it foregrounds systems that emit and revise programs and treats weight-based policies (§3.2) and reinforcement-learning skill discovery (§3.4) as context rather than as the object of the taxonomy. A survey centred on representation learning or on real-world data collection would draw the map differently. Second, the frontier we emphasise (§3.1.5) is populated by very recent, in some cases concurrent, systems; benchmark numbers across them are not directly comparable, and we therefore report capabilities qualitatively rather than ranking methods. Third, the field moves quickly: several 2025–2026 systems we cite were released while this survey was being written, and the coverage should be read as a snapshot. Finally, the "skill economy" framing (§4) is grounded in an emerging commercial trend (one vendor's marketplace at the time of writing); we treat it as motivation for open problems, not as a mature body of results.

6 FUTURE DIRECTIONS

Three directions follow directly from the map. **(1) Standardised evaluation of self-improvement.** The systems on the top rung (§3.1.5) each report improvement on their own splits; the field lacks a shared protocol that measures held-out success as a function of accumulated experience (an area-under-the-learning-curve metric), cross-embodiment transfer, and library-reuse rate under a common budget. Building such a benchmark is the natural companion to this survey. **(2) Adaptation as a first-class marketplace primitive.** The open problems of §4 suggest a research programme in which a downloaded skill is not a frozen artefact but a starting point that the §3.1 loop specialises to the local robot and environment—turning distribution (layer 2) and adaptation (layer 3) into a single pipeline. **(3) Bridging the RL and code flavours of skills.** §3.4 shows that skills are learned either as latent RL policies or as retrievable code; a unifying interface—code that wraps, calls, and refines learned policies, and learned policies distilled back into inspectable code—would let the self-improvement machinery of §3.1 operate over both. We see the combination of a standard self-improvement benchmark with an adaptation-centric marketplace as the most likely path from today's static skill stores to genuinely capable, compounding robots.

7 CONCLUSION

Robot learning is organising itself around a single question: should competence be shipped as *weights* or as *skills*? This survey took that question as its axis. We mapped the field into six branches (§2), positioned end-to-end vision-language- action models as the weights-based foil (§3.2), and gave the code-as-policy camp its own deep-dive (§3.1) organised by *degree of self-improvement*: from one-shot program synthesis, through within-task repair, persistent skill memory, and population search, to the open-ended loop that combines all three. That top cell—feedback + memory + search—is the least populated and least surveyed region of the field, and it is where systems such as ASPIRE and ENPIRE now sit. We complemented this with a map of how skill *repertoires* are built (§3.4), showing that skills come in an RL flavour and a code flavour of which only the latter self-improves without gradients, and we connected the academic taxonomy to the nascent skill economy (§4), where commercial marketplaces already distribute skills but ship only static playback. The gap between that static distribution and genuine on-device adaptation is, we argue, the defining research opportunity for physical AI, and the self-improvement ladder of §3.1 is the road across it.

REFERENCES

[1] Anurag Ajay, Aviral Kumar, Pulkit Agrawal, Sergey Levine, and Ofir Nachum. 2021. OPAL: Offline Primitive Discovery for Accelerating Offline Reinforcement Learning. In *International Conference on Learning Representations (ICLR)*. arXiv:2010.13611.

[2] Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Manuel Y. Galliker, Dibya Ghosh, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Devin LeBlanc, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Allen Z. Ren, Lucy Xiaoyang Shi, Laura Smith, Jost Tobias Springenberg, Kyle Stachowicz, James Tanner, Quan Vuong, Homer Walke, Anna Walling, Haohuan Wang, Lili Yu, and Ury Zhilinsky. 2025. $\pi_{0.5}$: A Vision-Language-Action Model with Open-World Generalization. *arXiv preprint* (2025). Physical Intelligence. arXiv:2504.16054.

[3] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Lucy Xiaoyang Shi, James Tanner, Quan Vuong, Anna Walling, Haohuan Wang, and Ury Zhilinsky. 2024. π_0: A Vision-Language-Action Flow Model for General Robot Control. *arXiv preprint* (2024). Physical Intelligence. arXiv:2410.24164.

[4] Konstantinos Bousmalis, Giulia Vezzani, Dushyant Rao, Coline Devin, Alex X. Lee, Maria Bauza, et al. 2023. RoboCat: A Self-Improving Generalist Agent for Robotic Manipulation. *Transactions on Machine Learning Research (TMLR)* (2023). arXiv:2306.11706.

[5] Anthony Brohan, Noah Brown, Justice Carbajal, et al. 2023. RT-1: Robotics Transformer for Real-World Control at Scale. In *Robotics: Science and Systems (RSS)*. arXiv:2212.06817.

[6] Anthony Brohan, Noah Brown, Justice Carbajal, et al. 2023. RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control. In *Conference on Robot Learning (CoRL)*. arXiv:2307.15818.

[7] Harold Haodong Chen, Sirui Chen, Yingjie Xu, Wenhang Ge, and Ying-Cong Chen. 2026. RoboEvolve: Co-Evolving Planner-Simulator for Robotic Manipulation with Limited Data. *arXiv preprint* (2026). arXiv:2605.13775.

[8] Junting Chen, Yao Mu, Qiaojun Yu, Tianming Wei, Silang Wu, Zhecheng Yuan, Zhixuan Liang, Chao Yang, Kaipeng Zhang, Wenqi Shao, Yu Qiao, Huazhe Xu, Mingyu Ding, and Ping Luo. 2024. RoboScript: Code Generation for Free-Form Manipulation Tasks across Real and Simulation. *arXiv preprint* (2024). arXiv:2402.14623.

[9] Lawrence Yunliang Chen, Kush Hari, Karthik Dharmarajan, Chenfeng Xu, Quan Vuong, and Ken Goldberg. 2024. Mirage: Cross-Embodiment Zero-Shot Policy Transfer with Cross-Painting. In *Robotics: Science and Systems (RSS)*. arXiv:2402.19249.

[10] Ria Doshi, Homer Walke, Oier Mees, Sudeep Dasari, and Sergey Levine. 2024. Scaling Cross-Embodied Learning: One Policy for Manipulation, Navigation, Locomotion and Aviation. In *Conference on Robot Learning (CoRL)*. arXiv:2408.11812.

[11] Jiafei Duan, Wilbert Pumacay, Nishanth Kumar, Yi Ru Wang, Shulin Tian, Wentao Yuan, Ranjay Krishna, Dieter Fox, Ajay Mandlekar, and Yijie Guo. 2024. AHA: A Vision-Language-Model for Detecting and Reasoning Over Failures in Robotic Manipulation. *arXiv preprint* (2024). arXiv:2410.00371.

[12] Benjamin Eysenbach, Abhishek Gupta, Julian Ibarz, and Sergey Levine. 2019. Diversity is All You Need: Learning Skills without a Reward Function. In *International Conference on Learning Representations (ICLR)*. arXiv:1802.06070.

[13] Letian Fu, Justin Yu, Karim El-Refai, Ethan Kou, Haoru Xue, Huang Huang, Wenli Xiao, Guanzhi Wang, Dantong Niu, Li Fei-Fei, Guanya Shi, Jiajun Wu, Shankar Sastry, Yuke Zhu, Ken Goldberg, and Linxi Fan. 2026. CaP-X: A Framework for Benchmarking and Improving Coding Agents for Robot Manipulation. *arXiv preprint* (2026). arXiv:2603.22435.

[14] Gemini Robotics Team, Google DeepMind. 2025. Gemini Robotics: Bringing AI into the Physical World. *arXiv preprint* (2025). arXiv:2503.20020.

[15] Dibya Ghosh, Homer Walke, Karl Pertsch, Kevin Black, Oier Mees, Sudeep Dasari, Joey Hejna, Tobias Kreiman, Charles Xu, Jianlan Luo, You Liang Tan, Lawrence Yunliang Chen, Pannag Sanketi, Quan Vuong, Ted Xiao, Dorsa Sadigh, Chelsea Finn, and Sergey Levine. 2024. Octo: An Open-Source Generalist Robot Policy. In *Robotics: Science and Systems (RSS)*. Octo Model Team. arXiv:2405.12213.

[16] Jiayuan Gu, Fanbo Xiang, Xuanlin Li, Zhan Ling, Xiqiang Liu, Tongzhou Mu, Yihe Tang, Stone Tao, Xinyue Wei, Yunchao Yao, Xiaodi Yuan, Pengwei Xie, Zhiao Huang, Rui Chen, and Hao Su. 2023. ManiSkill2: A Unified Benchmark for Generalizable Manipulation Skills. In *International Conference on Learning Representations (ICLR)*. arXiv:2302.04659.

[17] Ping Guo, Chao Li, Yinglan Feng, and Chaoning Zhang. 2026. Code Evolution for Control: Synthesizing Policies via LLM-Driven Evolutionary Search. *arXiv preprint* (2026). arXiv:2601.06845.

[18] Yanjiang Guo, Yen-Jen Wang, Lihan Zha, and Jianyu Chen. 2024. DoReMi: Grounding Language Model by Detecting and Recovering from Plan-Execution Misalignment. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. arXiv:2307.00329.

[19] Siyuan Huang, Zhengkai Jiang, Hao Dong, Yu Qiao, Peng Gao, and Hongsheng Li. 2023. Instruct2Act: Mapping Multi-modality Instructions to Robotic Actions with Large Language Model. *arXiv preprint* (2023). arXiv:2305.11176.

[20] Wenlong Huang, Chen Wang, Ruohan Zhang, Yunzhu Li, Jiajun Wu, and Li Fei-Fei. 2023. VoxPoser: Composable 3D Value Maps for Robotic Manipulation with Language Models. In *Conference on Robot Learning (CoRL)*. arXiv:2307.05973.

[21] Wenlong Huang, Fei Xia, Ted Xiao, Harris Chan, Jacky Liang, Pete Florence, Andy Zeng, Jonathan Tompson, Igor Mordatch, Yevgen Chebotar, Pierre Sermanet, Noah Brown, Tomas Jackson, Linda Luu, Sergey Levine, Karol Hausman, and Brian Ichter. 2022. Inner Monologue: Embodied Reasoning through Planning with Language Models. In *Conference on Robot Learning (CoRL)*. arXiv:2207.05608.

[22] Stephen James, Zicong Ma, David Rovick Arrojo, and Andrew J. Davison. 2020. RLBench: The Robot Learning Benchmark and Learning Environment. *IEEE Robotics and Automation Letters (RA-L)* (2020). arXiv:1909.12271.

[23] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. 2024. OpenVLA: An Open-Source Vision-Language-Action Model. In *Conference on Robot Learning (CoRL)*. arXiv:2406.09246.

[24] Michael Laskin, Hao Liu, Xue Bin Peng, Denis Yarats, Aravind Rajeswaran, and Pieter Abbeel. 2022. CIC: Contrastive Intrinsic Control for Unsupervised Skill Discovery. *arXiv preprint* (2022). arXiv:2202.00161.

[25] Chengshu Li, Ruohan Zhang, Josiah Wong, Cem Gokmen, Sanjana Srivastava, Roberto Martín-Martín, et al. 2024. BEHAVIOR-1K: A Human-Centered, Embodied AI Benchmark with 1,000 Everyday Activities and Realistic Simulation. *arXiv preprint* (2024). arXiv:2403.09227.

[26] Hao Li, Xue Yang, Zhaokai Wang, Xizhou Zhu, Jie Zhou, Yu Qiao, Xiaogang Wang, Hongsheng Li, Lewei Lu, and Jifeng Dai. 2024. Auto MC-Reward: Automated Dense Reward Design with Large Language Models for Minecraft. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. arXiv:2312.09238.

[27] Jingyao Li, Pengguang Chen, Sitong Wu, Chuanyang Zheng, Hong Xu, and Jiaya Jia. 2024. RoboCoder: Robotic Learning from Basic Skills to General Tasks with Large Language Models. *arXiv preprint* (2024). arXiv:2406.03757.

[28] Qixiu Li, Yaobo Liang, Zeyu Wang, Lin Luo, Xi Chen, Mozheng Liao, Fangyun Wei, Yu Deng, Sicheng Xu, Yizhong Zhang, Xiaofan Wang, Bei Liu, Jianlong Fu, Jianmin Bao, Dong Chen, Yuanchun Shi, Jiaolong Yang, and Baining Guo. 2024. CogACT: A Foundational Vision-Language-Action Model for Synergizing Cognition and Action in Robotic Manipulation. *arXiv preprint* (2024). arXiv:2411.19650.

[29] Ruiying Li, Yunlang Zhou, YuYao Zhu, Kylin Chen, Jingyuan Wang, Sukai Wang, Kongtao Hu, Minhui Yu, Bowen Jiang, Zhan Su, Jiayao Ma, Xin He, Yongjian Shen, Yang Yang, Guanghui Ren, Maoqing Yao, Wenhao Wang, and Yao Mu. 2026. RoboClaw: An Agentic Framework for Scalable Long-Horizon Robotic Tasks. *arXiv preprint* (2026). arXiv:2603.11558.

[30] Xinghang Li, Minghuan Liu, Hanbo Zhang, Cunjun Yu, Jie Xu, Hongtao Wu, Chilam Cheang, Ya Jing, Weinan Zhang, Huaping Liu, Hang Li, and Tao Kong. 2024. Vision-Language Foundation Models as Effective Robot Imitators. In *International Conference on Learning Representations (ICLR)*. RoboFlamingo. arXiv:2311.01378.

[31] Zaijing Li, Yuquan Xie, Rui Shao, Gongwei Chen, Dongmei Jiang, and Liqiang Nie. 2024. Optimus-1: Hybrid Multimodal Memory Empowered Agents Excel in Long-Horizon Tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2408.03615.

[32] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. 2023. Code as Policies: Language Model Programs for Embodied Control. In *IEEE International Conference on Robotics and Automation (ICRA)*. arXiv:2209.07753.

[33] Kaiqu Liang, Zixu Zhang, and Jaime Fernández Fisac. 2024. Introspective Planning: Aligning Robots' Uncertainty with Inherent Task Ambiguity. *arXiv preprint* (2024). arXiv:2402.06529.

[34] William Liang, Sam Wang, Hung-Ju Wang, Osbert Bastani, Dinesh Jayaraman, and Yecheng Jason Ma. 2024. Eureka-verse: Environment Curriculum Generation via Large Language Models. In *Conference on Robot Learning (CoRL)*. arXiv:2411.01775.

[35] Kevin Lin, Christopher Agia, Toki Migimatsu, Marco Pavone, and Jeannette Bohg. 2023. Text2Motion: From Natural Language Instructions to Feasible Plans. *Autonomous Robots* (2023). arXiv:2303.12153.

[36] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. 2023. LIBERO: Benchmarking Knowledge Transfer for Lifelong Robot Learning. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*. arXiv:2306.03310.

[37] Shunyu Liu, Yaoru Li, Kongcheng Zhang, Zhenyu Cui, Wenkai Fang, Yuxuan Zheng, Tongya Zheng, and Mingli Song. 2024. Odyssey: Empowering Minecraft Agents with Open-World Skills. *arXiv preprint* (2024). arXiv:2407.15325.

[38] Zeyi Liu, Arpit Bahety, and Shuran Song. 2023. REFLECT: Summarizing Robot Experiences for Failure Explanation and Correction. In *Conference on Robot Learning (CoRL)*. arXiv:2306.15724.

[39] Runyu Lu, Yubo Wu, Ethan Kou, Letian Fu, Wenli Xiao, Ajay Mandlekar, Yinzhen Xu, Guanya Shi, Ken Goldberg, Ang Chen, Mosharaf Chowdhury, Yuke Zhu, Linxi Fan, and Guanzhi Wang. 2026. ASPIRE: Agentic Skills Discovery for Robotics. *arXiv preprint* (2026). arXiv:2607.00272.

[40] Yecheng Jason Ma, William Liang, Guanzhi Wang, De-An Huang, Osbert Bastani, Dinesh Jayaraman, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2024. Eureka: Human-Level Reward Design via Coding Large Language Models. In *International Conference on Learning Representations (ICLR)*. arXiv:2310.12931.

[41] Yecheng Jason Ma, William Liang, Hung-Ju Wang, Sam Wang, Yuke Zhu, Linxi Fan, Osbert Bastani, and Dinesh Jayaraman. 2024. DrEureka: Language Model Guided Sim-To-Real Transfer. In *Robotics: Science and Systems (RSS)*. arXiv:2406.01967.

[42] Oier Mees, Lukas Hermann, Erick Rosete-Beas, and Wolfram Burgard. 2022. CALVIN: A Benchmark for Language-Conditioned Policy Learning for Long-Horizon Robot Manipulation Tasks. *IEEE Robotics and Automation Letters (RA-L)* (2022). arXiv:2112.03227.

[43] Yao Mu, Junting Chen, Qinglong Zhang, Shoufa Chen, Qiaojun Yu, Chongjian Ge, Runjian Chen, Zhixuan Liang, Mengkang Hu, Chaofan Tao, Peize Sun, Haibao Yu, Chao Yang, Wenqi Shao, Wenhai Wang, Jifeng Dai, Yu Qiao, Mingyu Ding, and Ping Luo. 2024. RoboCodeX: Multimodal Code Generation for Robotic Behavior Synthesis. In *International Conference on Machine Learning (ICML)*. arXiv:2402.16117.

[44] NVIDIA, Johan Bjorck, Fernando Castañeda, Nikita Cherniadev, Xingye Da, Runyu Ding, Linxi Fan, Yu Fang, Dieter Fox, Fengyuan Hu, et al. 2025. GR00T N1: An Open Foundation Model for Generalist Humanoid Robots. *arXiv preprint* (2025). NVIDIA. arXiv:2503.14734.

[45] Open X-Embodiment Collaboration. 2024. Open X-Embodiment: Robotic Learning Datasets and RT-X Models. In *IEEE International Conference on Robotics and Automation (ICRA)*. arXiv:2310.08864.

[46] Seohong Park, Jongwook Choi, Jaekyeom Kim, Honglak Lee, and Gunhee Kim. 2022. Lipschitz-constrained Unsupervised Skill Discovery. In *International Conference on Learning Representations (ICLR)*. arXiv:2202.00914.

[47] Seohong Park, Oleh Rybkin, and Sergey Levine. 2024. METRA: Scalable Unsupervised RL with Metric-Aware Abstraction. In *International Conference on Learning Representations (ICLR)*. arXiv:2310.08887.

[48] Karl Pertsch, Youngwoon Lee, and Joseph J. Lim. 2020. Accelerating Reinforcement Learning with Learned Skill Priors. In *Conference on Robot Learning (CoRL)*. arXiv:2010.11944.

[49] Delin Qu, Haoming Song, Qizhi Chen, Yuanqi Yao, Xinyi Ye, Yan Ding, Zhigang Wang, JiaYuan Gu, Bin Zhao, Dong Wang, and Xuelong Li. 2025. SpatialVLA: Exploring Spatial Representations for Visual-Language-Action Model. *arXiv preprint* (2025). arXiv:2501.15830.

[50] Archit Sharma, Shixiang Gu, Sergey Levine, Vikash Kumar, and Karol Hausman. 2020. Dynamics-Aware Unsupervised Discovery of Skills. In *International Conference on Learning Representations (ICLR)*. arXiv:1907.01657.

[51] Lucy Xiaoyang Shi, Joseph J. Lim, and Youngwoon Lee. 2022. Skill-based Model-based Reinforcement Learning. In *Conference on Robot Learning (CoRL)*. arXiv:2207.07560.

[52] Avi Singh, Huihan Liu, Gaoyue Zhou, Albert Yu, Nicholas Rhinehart, and Sergey Levine. 2021. Parrot: Data-Driven Behavioral Priors for Reinforcement Learning. In *International Conference on Learning Representations (ICLR)*. arXiv:2011.10024.

[53] Ishika Singh, Valts Blukis, Arsalan Mousavian, Ankit Goyal, Danfei Xu, Jonathan Tremblay, Dieter Fox, Jesse Thomason, and Animesh Garg. 2023. ProgPrompt: Generating Situated Robot Task Plans using Large Language Models. In *IEEE International Conference on Robotics and Automation (ICRA)*. arXiv:2209.11302.

[54] Georgios Tziafas and Hamidreza Kasaei. 2024. Lifelong Robot Library Learning: Bootstrapping Composable and Generalizable Skills for Embodied Control with Language Models. In *IEEE International Conference on Robotics and Automation (ICRA)*. arXiv:2406.18746.

[55] Unitree Robotics. 2026. UniStore: A Humanoid Robot Application Store. https://unistore.unitree.com.

[56] Sai Vemprala, Rogerio Bonatti, Arthur Bucker, and Ashish Kapoor. 2023. ChatGPT for Robotics: Design Principles and Model Abilities. *arXiv preprint* (2023). arXiv:2306.17582.

[57] Weikang Wan, Yifeng Zhu, Rutav Shah, and Yuke Zhu. 2024. LOTUS: Continual Imitation Learning for Robot Manipulation Through Unsupervised Skill Discovery. In *IEEE International Conference on Robotics and Automation (ICRA)*. arXiv:2311.02058.

[58] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023. Voyager: An Open-Ended Embodied Agent with Large Language Models. *arXiv preprint* (2023). arXiv:2305.16291.

[59] Huaxiaoyue Wang, Gonzalo Gonzalez-Pumariega, Yash Sharma, and Sanjiban Choudhury. 2023. Demo2Code: From Summarizing Demonstrations to Synthesizing Code via Extended Chain-of-Thought. In *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2305.16744.

[60] Yufei Wang, Zhou Xian, Feng Chen, Tsun-Hsuan Wang, Yian Wang, Katerina Fragkiadaki, Zackory Erickson, David Held, and Chuang Gan. 2024. RoboGen: Towards Unleashing Infinite Data for Automated Robot Learning via Generative Simulation. In *International Conference on Machine Learning (ICML)*. arXiv:2311.01455.

[61] Yen-Jen Wang, Bike Zhang, Jianyu Chen, and Koushil Sreenath. 2024. Prompt a Robot to Walk with Large Language Models. In *IEEE Conference on Decision and Control (CDC)*. arXiv:2309.09969.

[62] Zihao Wang, Shaofei Cai, Anji Liu, Yonggang Jin, Jinbing Hou, Bowei Zhang, Haowei Lin, Zhaofeng He, Zilong Zheng, Yaodong Yang, Xiaojian Ma, and Yitao Liang. 2023. JARVIS-1: Open-World Multi-task Agents with Memory-Augmented Multimodal Language Models. *arXiv preprint* (2023). arXiv:2311.05997.

[63] Jimmy Wu, Rika Antonova, Adam Kan, Marion Lepert, Andy Zeng, Shuran Song, Jeannette Bohg, Szymon Rusinkiewicz, and Thomas Funkhouser. 2023. TidyBot: Personalized Robot Assistance with Large Language Models. *Autonomous Robots* (2023). arXiv:2305.05658.

[64] Wenli Xiao, Jia Xie, Tonghe Zhang, Haotian Lin, Letian Fu, Haoru Xue, Jalen Lu, Yi Yang, Cunxi Dai, Zi Wang, Jimmy Wu, Guanzhi Wang, S. Shankar Sastry, Ken Goldberg, Linxi Fan, Yuke Zhu, and Guanya Shi. 2026. ENPIRE: Agentic Robot Policy Self-Improvement in the Real World. *arXiv preprint* (2026). arXiv:2606.19980.

[65] Senwei Xie, Hongyu Wang, Zhanqi Xiao, Ruiping Wang, and Xilin Chen. 2025. Robotic Programmer: Video Instructed Policy Code Generation for Robotic Manipulation. *arXiv preprint* (2025). arXiv:2501.04268.

[66] Senwei Xie, Yuntian Zhang, Ruiping Wang, and Xilin Chen. 2026. Uni-Skill: Building Self-Evolving Skill Repository for Generalizable Robotic Manipulation. In *IEEE International Conference on Robotics and Automation (ICRA)*. arXiv:2603.02623.

[67] Tianbao Xie, Siheng Zhao, Chen Henry Wu, Yitao Liu, Qian Luo, Victor Zhong, Yanchao Yang, and Tao Yu. 2024. Text2Reward: Reward Shaping with Language Models for Reinforcement Learning. In *International Conference on Learning Representations (ICLR)*. arXiv:2309.11489.

[68] Takuma Yoneda, Jiading Fang, Peng Li, Huanyu Zhang, Tianchong Jiang, Shengjie Lin, Ben Picker, David Yunis, Hongyuan Mei, and Matthew R. Walter. 2024. Statler: State-Maintaining Language Models for Embodied Reasoning. In *IEEE International Conference on Robotics and Automation (ICRA)*. arXiv:2306.17840.

[69] Tianhe Yu, Deirdre Quillen, Zhanpeng He, Ryan Julian, Avnish Narayan, Hayden Shively, Adithya Bellathur, Karol Hausman, Chelsea Finn, and Sergey Levine. 2019. Meta-World: A Benchmark and Evaluation for Multi-Task and Meta Reinforcement Learning. In *Conference on Robot Learning (CoRL)*. arXiv:1910.10897.

[70] Wenhao Yu, Nimrod Gileadi, Chuyuan Fu, Sean Kirmani, Kuang-Huei Lee, Montse Gonzalez Arenas, Hao-Tien Lewis Chiang, Tom Erez, Leonard Hasenclever, Jan Humplik, Brian Ichter, Ted Xiao, Peng Xu, Andy Zeng, Tingnan Zhang, Nicolas Heess, Dorsa Sadigh, Jie Tan, Yuval Tassa, and Fei Xia. 2023. Language to Rewards for Robotic Skill Synthesis. In *Conference on Robot Learning (CoRL)*. arXiv:2306.08647.

[71] Lihan Zha, Yuchen Cui, Li-Heng Lin, Minae Kwon, Montserrat Gonzalez Arenas, Andy Zeng, Fei Xia, and Dorsa Sadigh. 2024. Distilling and Retrieving Generalizable Knowledge for Robot Manipulation via Language Corrections. In *IEEE International Conference on Robotics and Automation (ICRA)*. arXiv:2311.10678.

[72] Jesse Zhang, Karl Pertsch, Jiahui Zhang, and Joseph J. Lim. 2024. SPRINT: Scalable Policy Pre-Training via Language Instruction Relabeling. In *IEEE International Conference on Robotics and Automation (ICRA)*. arXiv:2306.11886.

[73] Jesse Zhang, Jiahui Zhang, Karl Pertsch, Ziyi Liu, Xiang Ren, Minsuk Chang, Shao-Hua Sun, and Joseph J. Lim. 2023. Bootstrap Your Own Skills: Learning to Solve New Tasks with Large Language Model Guidance. In *Conference on Robot Learning (CoRL)*. arXiv:2310.10021.

[74] Ziao Zhang, Kou Shi, Shiting Huang, Avery Nie, Yu Zeng, Yiming Zhao, Zhen Fang, Qisheng Su, Haibo Qiu, Wei Yang, Qingnan Ren, Shun Zou, Wenxuan Huang, Lin Chen, Zehui Chen, and Feng Zhao. 2026. SkillFlow: Benchmarking Lifelong Skill Discovery and Evolution for Autonomous Agents. *arXiv preprint* (2026). arXiv:2604.17308.

[75] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. ExpeL: LLM Agents Are Experiential Learners. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*. arXiv:2308.10144.

[76] Enshen Zhou, Qi Su, Cheng Chi, Zhizheng Zhang, Zhongyuan Wang, Tiejun Huang, Lu Sheng, and He Wang. 2025. Code-as-Monitor: Constraint-aware Visual Programming for Reactive and Proactive Robotic Failure Detection. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. arXiv:2412.04455.

[77] Xizhou Zhu, Yuntao Chen, Hao Tian, Chenxin Tao, Weijie Su, Chenyu Yang, Gao Huang, Bin Li, Lewei Lu, Xiaogang Wang, Yu Qiao, Zhaoxiang Zhang, and Jifeng Dai. 2023. Ghost in the Minecraft: Generally Capable Agents for Open-World Environments via Large Language Models with Text-based Knowledge and Memory. *arXiv preprint* (2023). arXiv:2305.17144.

[78] Yuke Zhu, Josiah Wong, Ajay Mandlekar, Roberto Martín-Martín, Abhishek Joshi, Kevin Lin, Abhiram Maddukuri, Soroush Nasiriany, and Yifeng Zhu. 2020. robosuite: A Modular Simulation Framework and Benchmark for Robot Learning. *arXiv preprint* (2020). arXiv:2009.12293.